Guía para la práctica 3

Analizadores sintácticos de dependencias basados en transiciones

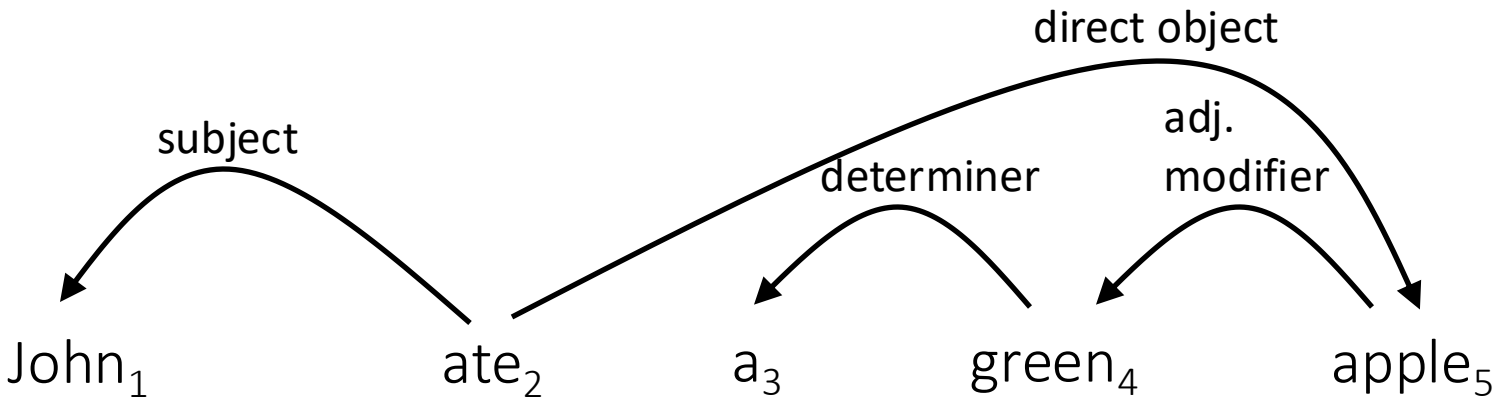
Fundamentos de Procesamiento del Lenguaje Natural

Grado en Inteligencia Artificial

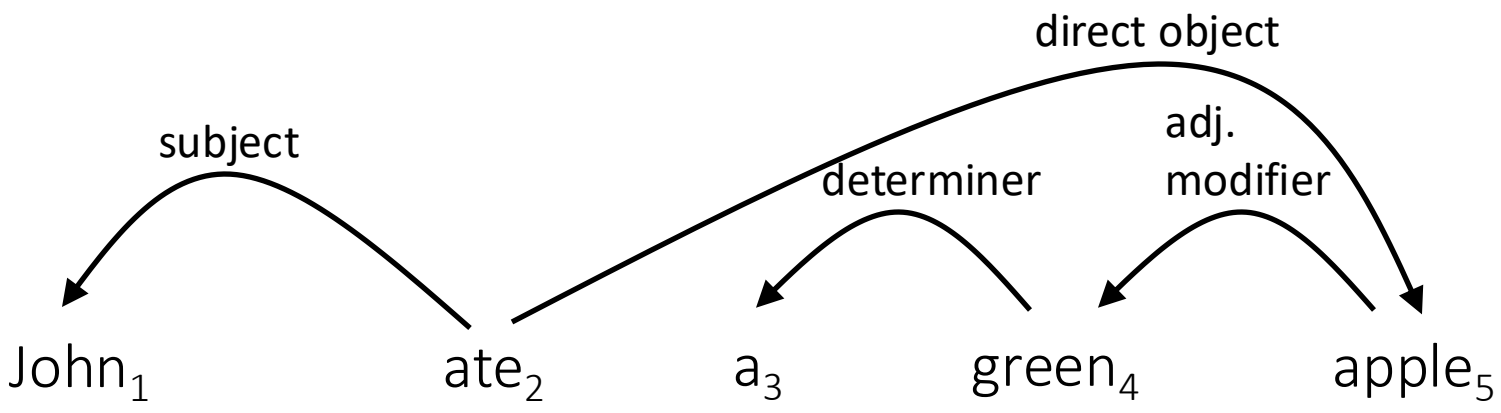
UNIVERSIDADE DA CORUÑA

Árboles sintácticos de dependencias

Informalmente, “qué hace qué a quién”:



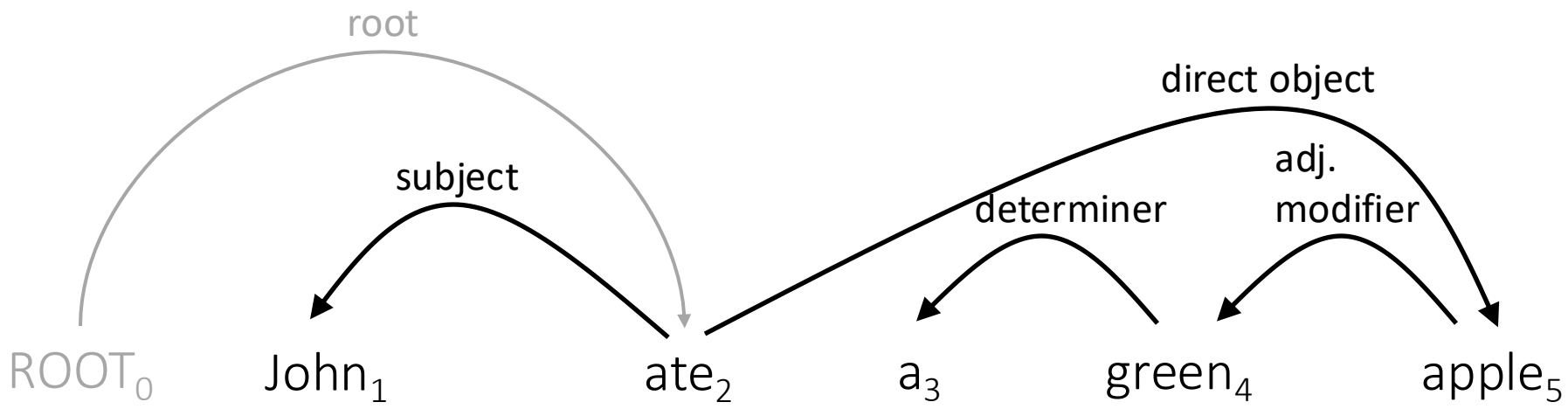
Árboles sintácticos de dependencias



Cada token tiene exactamente un padre (por ejemplo, ‘ate’ es el padre de ‘John’)

Cada token puede tener cero, uno o varios hijos (por ejemplo, ‘ate’ tiene dos hijos, el propio ‘John’ y el token ‘apple’; mientras que ‘a’ no tiene ninguno).

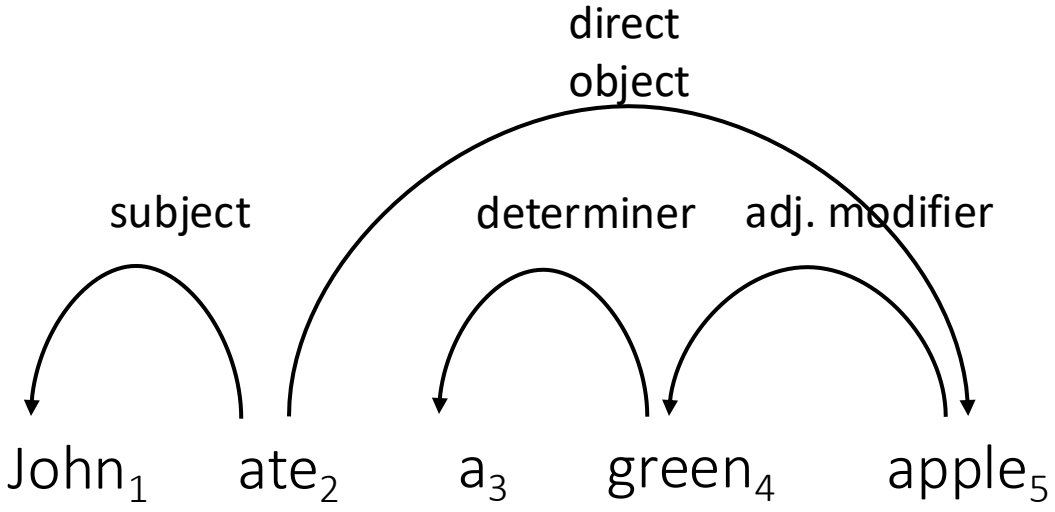
Árboles sintácticos de dependencias



Suele incluirse un token con ID 0, denominado habitualmente *dummy root* para marcar qué token es la verdadera raíz de la oración.

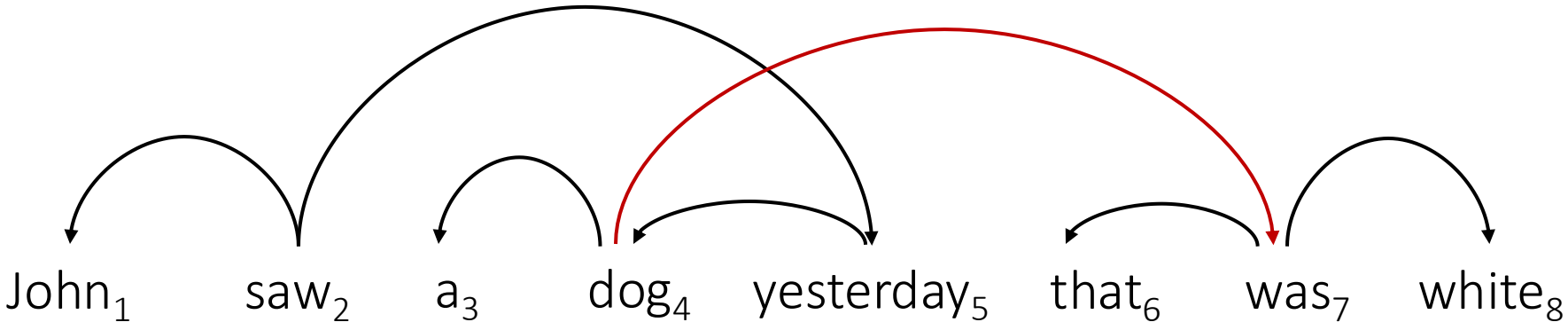
Árboles sintácticos de dependencias

Árboles proyectivos
(es posible dibujar el árbol sin que los arcos se crucen)



Árboles sintácticos de dependencias

Árboles no proyectivos
(no es posible dibujar el árbol sin que los arcos se crucen)



Treebanks disponibles

https://universaldependencies.org/ (Universal Dependencies, UD)

Treebanks disponibles para más de 100 idiomas.

Para la representación de los árboles de dependencias, utiliza un formato denominado CoNLL-U.

```
# sent_id = weblog-juancole.com_juancole_20051126063000_ENG_20051126_063000-0028
# text = This item is a small one and easily missed.
1	This	this	DET	DT	Number=Sing|PronType=Dem	2	det	2:det	_
2	item	item	NOUN	NN	Number=Sing	6	nsubj	6:nsubj|9:nsubj:pass	_
3	is	be	AUX	VBZ	Mood=Ind|Number=Sing|Person=3|Tense=Pres|VerbForm=Fin	6	cop	6:cop	_
4	a	a	DET	DT	Definite=Ind|PronType=Art	6	det	6:det	_
5	small	small	ADJ	JJ	Degree=Pos	6	amod	6:amod	_
6	one	one	NOUN	NN	Number=Sing	0	root	0:root	_
7	and	and	CCONJ	CC	_	9	cc	9:cc	_
8	easily	easily	ADV	RB	_	9	advmod	9:advmod	_
9	missed	miss	VERB	VBN	Tense=Past|VerbForm=Part	6	conj	6:conj:and	SpaceAfter=No
10	.	.	PUNCT	.	_	6	punct	6:punct	_
```

Treebanks disponibles

1. ID: Índice de la palabra dentro de la oración. Comienza en 1. Cuidado! El formato ConLL-U contiene también información sobre expresiones multi-palabra y nodos vacíos, representados como X.Y o X-Y (y que se deben ignorar para tareas de análisis sintáctico de dependencias).
2. FORM: La propia palabra en su forma original.
3. LEMMA: El lema o raíz de la palabra.
4. UPOS: Etiqueta con información morfológica universal (mismo conjunto de etiquetas en todos los treebanks de UD).
5. XPOS: Etiqueta con información morfológica específica del idioma.
6. FEATS: Información morfológica extra o de grano más fino.
7. HEAD: Padre de la palabra actual, bien con un valor de ID o cero (para indicar que es la raíz verdadera de la oración).
8. DEPREL: Tipo de dependencia entre la palabra y su HEAD.
9. DEPS: Información sobre enhanced dependency graphs (no se usa para análisis sintáctico de dependencias).
10. MISC: Otra información extra.

**Si para un campo no se dispone de información, se rellena con el carácter _*

CoNLLU Tokens

Para almacenar la información se proporciona, ya implementada, la clase Token dentro de conllu_token.py. Es una clase muy simple, que simplemente guarda dentro de un objeto toda la información de la palabra.

```
class Token():
    """
    A class to encode a line from a CoNLLU file.

    Attributes:
    - id (int): Token ID in the sentence.
    - form (str): The form or orthography of the word.
    - lemma (str): The lemma or base form of the word.
    - upos (str): Universal part-of-speech tag.
    - cpos (str): Language-specific part-of-speech tag; not used in this assignment.
    - feats (str): List of morphological features from the universal feature inventory or language-specific extension; separated by '|'.
    - head (int): Head of the current token, which is either a value of ID or zero ('0').
    - dep (str): Universal dependency relation to the HEAD.
    - deps (str): Enhanced dependency graph in the form of a list of head-deprel pairs.
    - misc (str): Any other annotation.

    Methods:
    - get_fields_list(): Returns a list of all the attribute values of the token, in the same order requited by the conllu format

    # Example usage:

    token_example = Token(1, "Distribution", "distribution", "NOUN", "S", "Number=Sing", 7, "nsubj")
    """
```

Lectura y escritura de ficheros

Ya está implementada en el fichero conllu_reader.py junto con otros materiales de esta práctica. No deberíais necesitar añadir nada más! Se incluyen ejemplos de uso al final del propio fichero.

```
class ConlluReader():
    """
    A class for reading and processing CoNLLU format files. CoNLLU (Conference on Natural Language
    Learning Universal Dependencies) format is widely used for linguistic annotation, especially in
    dependency parsing. This class provides methods to convert between tree structures (as a list of
    Token objects) and CoNLLU formatted strings, as well as to read CoNLLU files.

    Methods:
        tree2conllustr(tree: list[Token]) -> str:
            Converts a tree structure (list of Token objects) into a CoNLLU formatted string.

        conllustr2tree(conllustr: str, inference: bool = True) -> list[Token]:
            Converts a CoNLLU formatted string into a list of Token objects representing a tree.

        read_conllu_file(path: str, inference: bool = False) -> list:
            Reads a CoNLLU file and converts it into a list of tree structures.

        write_conllu_file(path: str, trees: list['Token']):
            Writes a list of tree structures to a file in CoNLLU format.

        remove_non_projective_trees(trees: list['Token']) -> list:
            Filters out non-projective trees from a list of tree structures.

    Note: Example usage is provided at the end of the file.
    """
```

Modelos basados en transiciones

Formalmente se representan como un tupla de la forma (C, T, c_s, C_t):

C: el conjunto de *estados.*

T: el conjunto de *transiciones* del algoritmo, una función parcial $t: C \rightarrow C$.

c_s: El estado inicial.

C_t : El conjunto de posible estados finales.

Modelos basados en transiciones

Hay varios algoritmos de transiciones disponibles.

Veréis varios de ellos en teoría en las próximas semanas.

En esta práctica vamos a implementar uno de los clásicos, el arc-eager.

Algoritmo arc-eager

La forma más habitual de representar un *estado* en un modelo de transiciones es con una tripleta de la forma (σ, β, A):

σ: Un *stack,* que almacena las palabras parcialmente procesadas. Estructura LIFO (Last In, First Out).

β: Un *buffer*, que almacena las palabras que aún quedan por leer. Estructura FIFO (First In, First Out).

A: El conjunto de arcos del árbol que han sido ya creados.

Para pasar de un estado c_i a un estado c_{i+1}, se aplican *transiciones*. El proceso se repite hasta alcanzar un denominado *estado final*.

Algoritmo arc-eager

La implementación del estado ya se da implementada como parte de la práctica, en la clase State dentro de state.py. Se incluyen ejemplos de uso al final del propio fichero.

```
class State(object):
    """
    Class to represent a parsing state in dependency parsing.

    Attributes:
        S (list['Token']): A stack holding tokens that are
                           currently being processed.
        B (list['Token']): A buffer holding tokens that are yet
                           to be processed.
        A (set[tuple]): A set of arcs of the form (head_id, dependency_label, dependent_id)
                        created during parsing, representing the dependencies.

    The class is used in dependency parsing algorithms to maintain the state of the parsing
    process, including which tokens are being considered and the relationships formed between tokens.
    """
```

Algoritmo arc-eager

Estado inicial:

σ: Inicialmente solo contiene el nodo ROOT, i.e., σ = [0]

β: Inicialmente contiene la secuencia completa que se usa como entrada i.e., β=[1,…,N]

A: Inicializado a vacío, ningún arco ha sido aún creado, i.e., A= {}

Estado final: cualquiera en el que β = [] (no importa que contenido haya en σ). Esto varía según el algoritmo de transiciones que usemos (lo veréis en teoría con más calma).

Algoritmo arc-eager

LEFT-ARC$_l$ – Crea un arco (con una etiqueta l) desde la primera palabra del buffer a la palabra actualmente en la cima del stack. A continuación, elimina la palabra de la cima del stack.

RIGHT-ARC$_l$ – Crea un arco (con una etiqueta l) desde la palabra en la cima del stack, a la primera palabra del buffer. A continuación, mueve la primera palabra del buffer a la cima del stack.

REDUCE – Elimina la palabra de la cima del stack (importante! Aplicar esta transición implica que para esa palabra ya hemos establecido todas sus relaciones).

SHIFT – Mueve la primera palabra del buffer a la cima del stack.

Algoritmo arc-eager

Para representar las transiciones también se proporciona la clasa Transition ya implementada, dentro del fichero algorithm.py. Se incluyen ejemplos de uso al final del fichero.

```
class Transition(object):
    """
    Class to represent a parsing transition in a dependency parser.

    Attributes:
    - action (str): The action to take, represented as an string constant. Actions include SHIFT, REDUCE, LEFT-ARC, or RIGHT-ARC.
    - dependency (str): The type of dependency relationship (only for LEFT-ARC and RIGHT-ARC, otherwise it'll be None), corresponding to the deprel column
    """
```

Algoritmo arc-eager

Acciones representadas como un sistema deductivo

$$\text{LEFT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma, j|\beta, A \cup \{(j, l, i)\}]}$$

$$\text{RIGHT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma|i|j, \beta, A \cup \{(i, l, j)\}]}$$

$$\text{REDUCE} = \frac{[\sigma|i, \beta, A]}{[\sigma, \beta, A]}$$

$$\text{SHIFT} = \frac{[\sigma, i|\beta, A]}{[\sigma|i, \beta, A]}$$

Precondiciones para que una operación sea válida

$$\text{LEFT-ARC} = \neg[i = 0]; \neg\exists k \exists l\ [(k, l, i) \in A]$$

$$\text{RIGHT-ARC} = \neg\exists k \exists l\ [(k, l, j) \in A]$$

$$\text{REDUCE} = \exists k \exists l [(k, l, i) \in A];$$

Algoritmo arc-eager

Acciones representadas como un sistema deductivo

$$\text{LEFT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma, j|\beta, A \cup \{(j, l, i)\}]}$$

$$\text{RIGHT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma|i|j, \beta, A \cup \{(i, l, j)\}]}$$

$$\text{REDUCE} = \frac{[\sigma|i, \beta, A]}{[\sigma, \beta, A]}$$

$$\text{SHIFT} = \frac{[\sigma, i|\beta, A]}{[\sigma|i, \beta, A]}$$

Precondiciones para que una operación sea válida

LEFT-ARC La palabra en la cima del stack no es el ROOT(0); y no se ha creado aún un arco donde la cima del stack (i) es el nodo hijo

RIGHT-ARC No se ha creado aún un arco donde el frente del buffer es el nodo hijo

REDUCE Ya se ha creado un arco creado donde la cima del stack (i) es el nodo hijo.

Modelo supervisado para el análisis sintáctico de dependencias

Como es habitual tendremos los conjuntos de entrenamiento, desarrollo y test.

El objetivo es, dada una oración, entrenar un modelo que obtenga la secuencia de transiciones que permita recuperar la estructura sintáctica de la oración.

Entrenamiento

Tenemos acceso al árbol de referencia, o árbol *gold*.

Derivamos la secuencia de transiciones que nos permitiría reconstruir el árbol.

Empezamos en un estado inicial. En *cada estado seleccionamos la siguiente transición que sea válida y correcta*.

Actualizamos el estado en función de la transición seleccionada.

Repetimos el proceso hasta alcanzar un estado final.

Esto es lo que se suele denominar como la implementación del *oráculo.*

Cada par (estado, transición) es una muestra de entrenamiento.

Entrenamiento - Ejemplo de oráculo

Precondiciones

LEFT-ARC La palabra en la cima del stack no es el ROOT(0); y no se ha creado aún un arco donde la cima del stack (i) es el nodo hijo

RIGHT-ARC No se ha creado aún un arco donde el frente del buffer es el nodo hijo

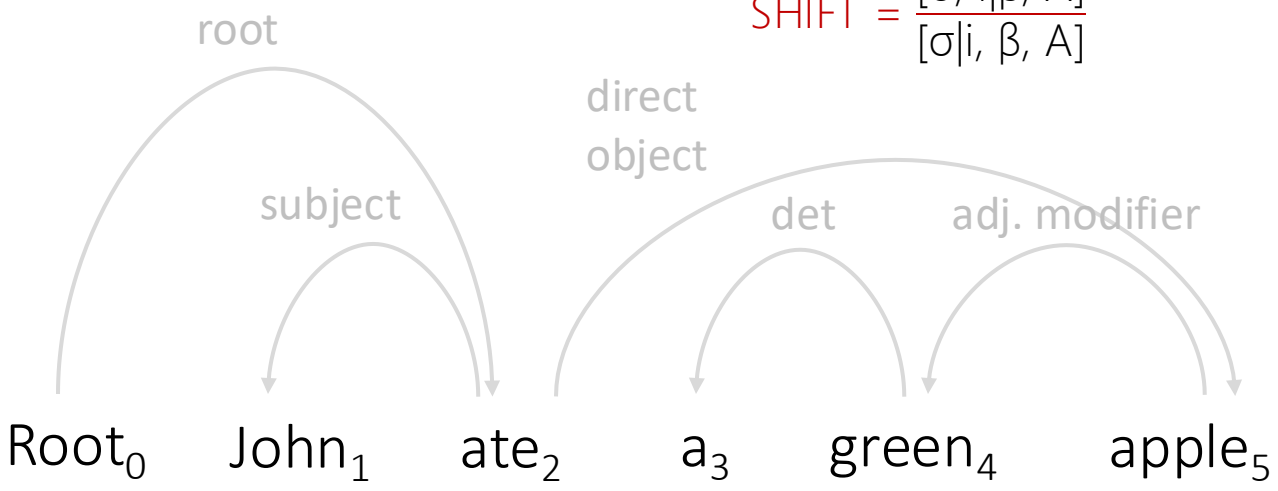
REDUCE Ya se ha creado un arco creado donde la cima del stack (i) es el nodo hijo.

$$\text{LEFT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma, j|\beta, A \cup \{(j, l, i)\}]}$$

$$\text{RIGHT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma|i|j, \beta, A \cup \{(i, l, j)\}]}$$

$$\text{REDUCE} = \frac{[\sigma|i, \beta, A]}{[\sigma, \beta, A]}$$

$$\text{SHIFT} = \frac{[\sigma, i|\beta, A]}{[\sigma|i, \beta, A]}$$



	Stack	Buffer	Action	Arcs
1	[0]	[1,2,3,4,5]	SHIFT	{}

En cada paso, analizamos y seleccionamos la transición que sea válida (cumple las precondiciones) y correcta (su aplicación contribuye a obtener el árbol objetivo)

LEFT-ARC? No es válida, la cima del stack es el ROOT (i=0)

RIGHT-ARC? Válida, pero incorrecta (no hay un arco ROOT -> John en el árbol)

REDUCE? No es válida, ROOT no es aún dependiente de ninguna palabra (en concreto el ROOT nunca se podrá reducir)

SHIFT? Válida y correcta.

Entrenamiento - Ejemplo de oráculo

Precondiciones

LEFT-ARC La palabra en la cima del stack no es el ROOT(0); y no se ha creado aún un arco donde la cima del stack (i) es el nodo hijo

RIGHT-ARC No se ha creado aún un arco donde el frente del buffer es el nodo hijo

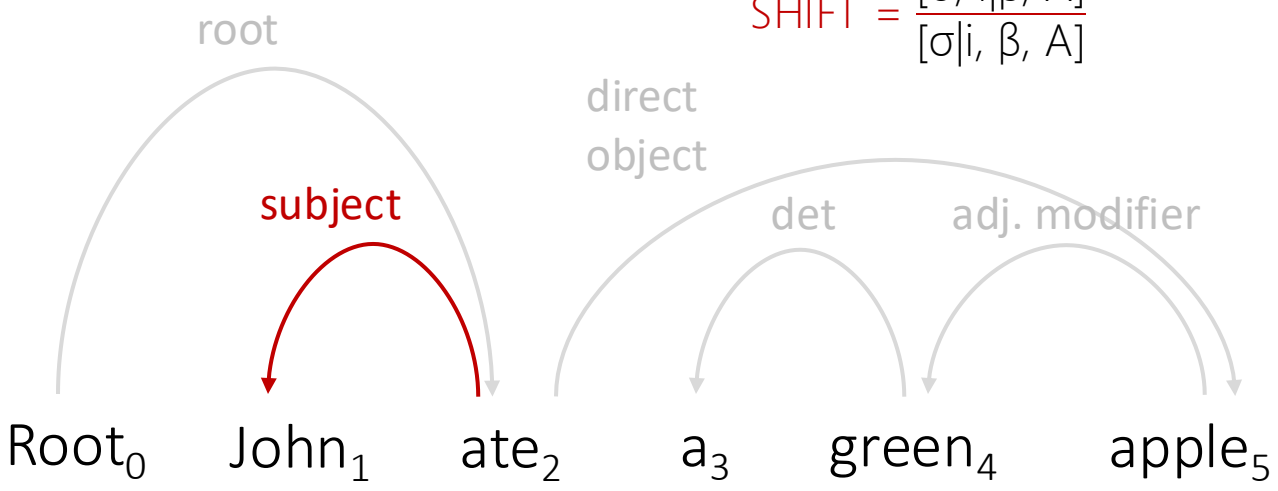
REDUCE Ya se ha creado un arco creado donde la cima del stack (i) es el nodo hijo.

$$\text{LEFT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma, j|\beta, A \cup \{(j, l, i)\}]}$$

$$\text{RIGHT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma|i|j, \beta, A \cup \{(i, l, j)\}]}$$

$$\text{REDUCE} = \frac{[\sigma|i, \beta, A]}{[\sigma, \beta, A]}$$

$$\text{SHIFT} = \frac{[\sigma, i|\beta, A]}{[\sigma|i, \beta, A]}$$



	Stack	Buffer	Action	Arcs
1	[0]	[1,2,3,4,5]	SHIFT	{}
2	[0,1]	[2,3,4,5]	LEFT-ARC	{(2,1)}

LEFT-ARC? Válida y correcta (hay un arco entre $John_1$ <- ate_2)

RIGHT-ARC? Válida, pero no correcta (no hay un arco $John_1$ -> ate_2)

REDUCE? No es válida: $John_1$ aún no ha sido asignado un padre.

SHIFT? Válida, pero incorrecta. Si la aplicásemos, ya no podríamos construir el arco $John_1$ <- ate_2, porque tanto 'John' como 'ate' pasarían a estar en el stack, y en el arc-eager nunca se crean arcos entre 2 palabras del stack.

Entrenamiento - Ejemplo de oráculo

Precondiciones

LEFT-ARC La palabra en la cima del stack no es el ROOT(0); y no se ha creado aún un arco donde la cima del stack (i) es el nodo hijo

RIGHT-ARC No se ha creado aún un arco donde el frente del buffer es el nodo hijo

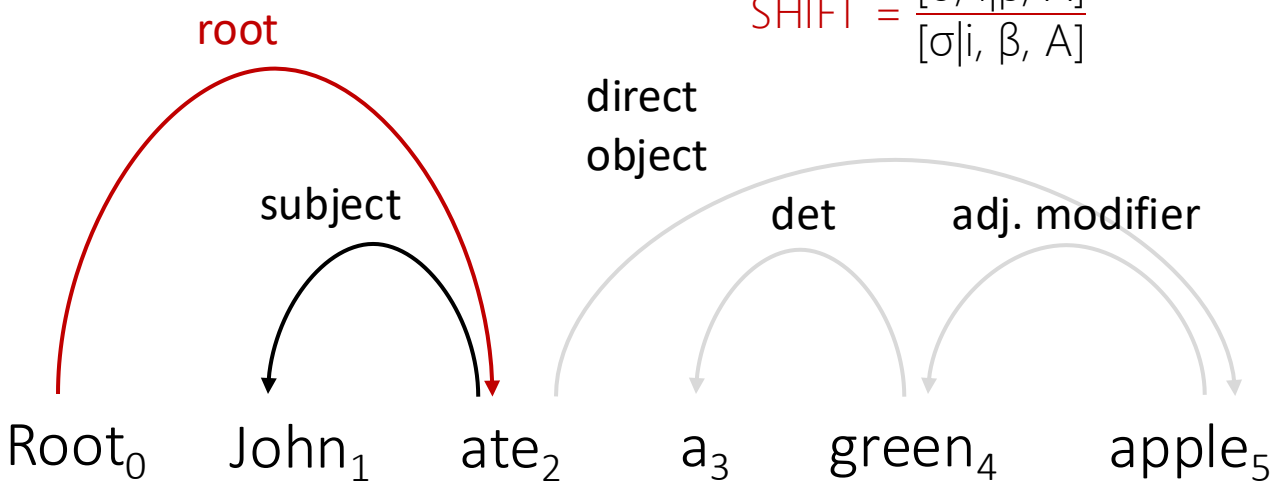
REDUCE Ya se ha creado un arco creado donde la cima del stack (i) es el nodo hijo.

$$\text{LEFT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma, j|\beta, A \cup \{(j, l, i)\}]}$$

$$\text{RIGHT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma|i|j, \beta, A \cup \{(i, l, j)\}]}$$

$$\text{REDUCE} = \frac{[\sigma|i, \beta, A]}{[\sigma, \beta, A]}$$

$$\text{SHIFT} = \frac{[\sigma, i|\beta, A]}{[\sigma|i, \beta, A]}$$



	Stack	Buffer	Action	Arcs
1	[0]	[1,2,3,4,5]	SHIFT	{}
2	[0,1]	[2,3,4,5]	LEFT-ARC	{(2,1)}
3	[0]	[2,3,4,5]	RIGHT-ARC	{(2,1),(0,2)}

** Por temas de espacio y claridad, no se indica en la columna Arcs el tipo de dependencia para el arco creado, simplificando como (2,1) el arco (2,subject,1), por ejemplo. En la implementación de la práctica sí es necesario incluir el tipo de dependencia.*

Entrenamiento - Ejemplo de oráculo

Precondiciones

LEFT-ARC La palabra en la cima del stack no es el ROOT(0); y no se ha creado aún un arco donde la cima del stack (i) es el nodo hijo

RIGHT-ARC No se ha creado aún un arco donde el frente del buffer es el nodo hijo

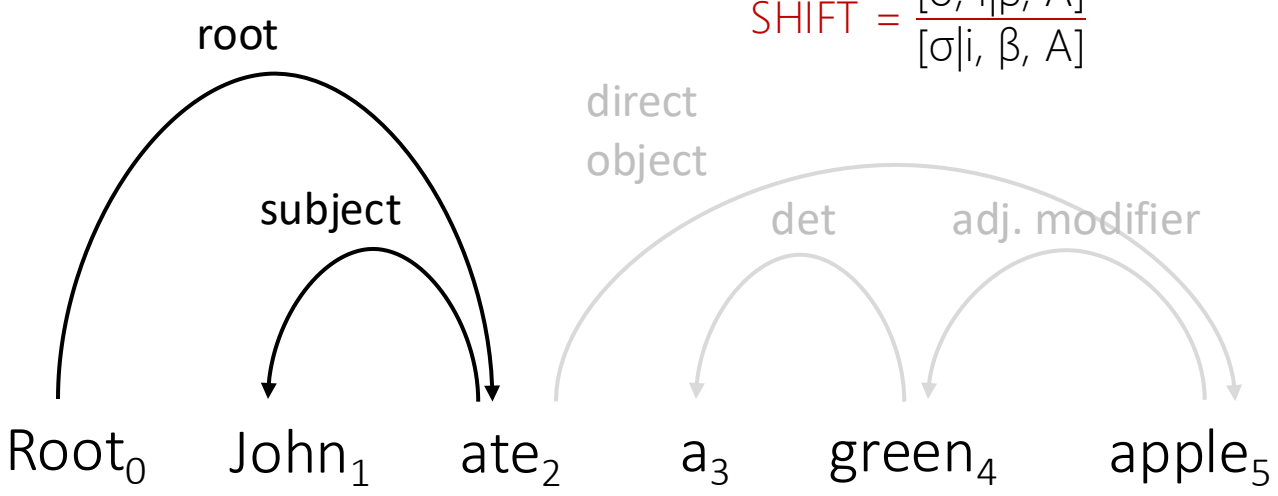
REDUCE Ya se ha creado un arco creado donde la cima del stack (i) es el nodo hijo.

$$\text{LEFT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma, j|\beta, A \cup \{(j, l, i)\}]}$$

$$\text{RIGHT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma|i|j, \beta, A \cup \{(i, l, j)\}]}$$

$$\text{REDUCE} = \frac{[\sigma|i, \beta, A]}{[\sigma, \beta, A]}$$

$$\text{SHIFT} = \frac{[\sigma, i|\beta, A]}{[\sigma|i, \beta, A]}$$



	Stack	Buffer	Action	Arcs
1	[0]	[1,2,3,4,5]	SHIFT	{}
2	[0,1]	[2,3,4,5]	LEFT-ARC	{(2,1)}
3	[0]	[2,3,4,5]	RIGHT-ARC	{(2,1),(0,2)}
4	[0,2]	[3,4,5]	SHIFT	{(2,1),(0,2)}

** Por temas de espacio y claridad, no se indica en la columna Arcs el tipo de dependencia para el arco creado, simplificando como (2,1) el arco (2,subject,1), por ejemplo. En la implementación de la práctica sí es necesario incluir el tipo de dependencia.*

Entrenamiento - Ejemplo de oráculo

Precondiciones

LEFT-ARC La palabra en la cima del stack no es el ROOT(0); y no se ha creado aún un arco donde la cima del stack (i) es el nodo hijo

RIGHT-ARC No se ha creado aún un arco donde el frente del buffer es el nodo hijo

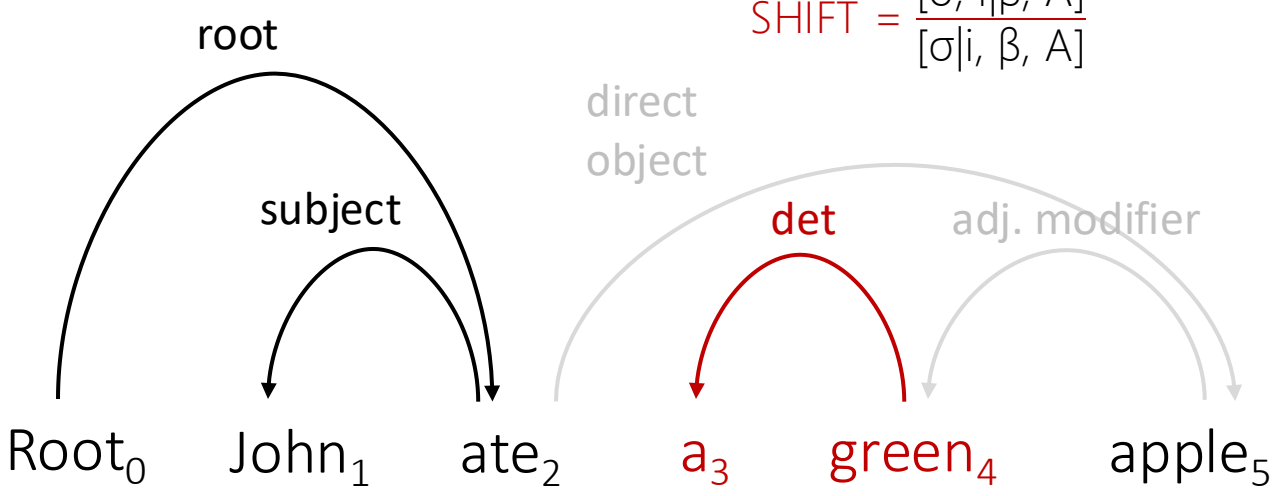
REDUCE Ya se ha creado un arco creado donde la cima del stack (i) es el nodo hijo.

$$\text{LEFT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma, j|\beta, A \cup \{(j, l, i)\}]}$$

$$\text{RIGHT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma|i|j, \beta, A \cup \{(i, l, j)\}]}$$

$$\text{REDUCE} = \frac{[\sigma|i, \beta, A]}{[\sigma, \beta, A]}$$

$$\text{SHIFT} = \frac{[\sigma, i|\beta, A]}{[\sigma|i, \beta, A]}$$



	Stack	Buffer	Action	Arcs
1	[0]	[1,2,3,4,5]	SHIFT	{}
2	[0,1]	[2,3,4,5]	LEFT-ARC	{(2,1)}
3	[0]	[2,3,4,5]	RIGHT-ARC	{(2,1),(0,2)}
4	[0,2]	[3,4,5]	SHIFT	{(2,1),(0,2)}
5	[0,2,3]	[4,5]	LEFT-ARC	{(2,1), (0,2), (4,3)}

** Por temas de espacio y claridad, no se indica en la columna Arcs el tipo de dependencia para el arco creado, simplificando como (2,1) el arco (2,subject,1), por ejemplo. En la implementación de la práctica sí es necesario incluir el tipo de dependencia.*

Entrenamiento - Ejemplo de oráculo

Precondiciones

LEFT-ARC La palabra en la cima del stack no es el ROOT(0); y no se ha creado aún un arco donde la cima del stack (i) es el nodo hijo

RIGHT-ARC No se ha creado aún un arco donde el frente del buffer es el nodo hijo

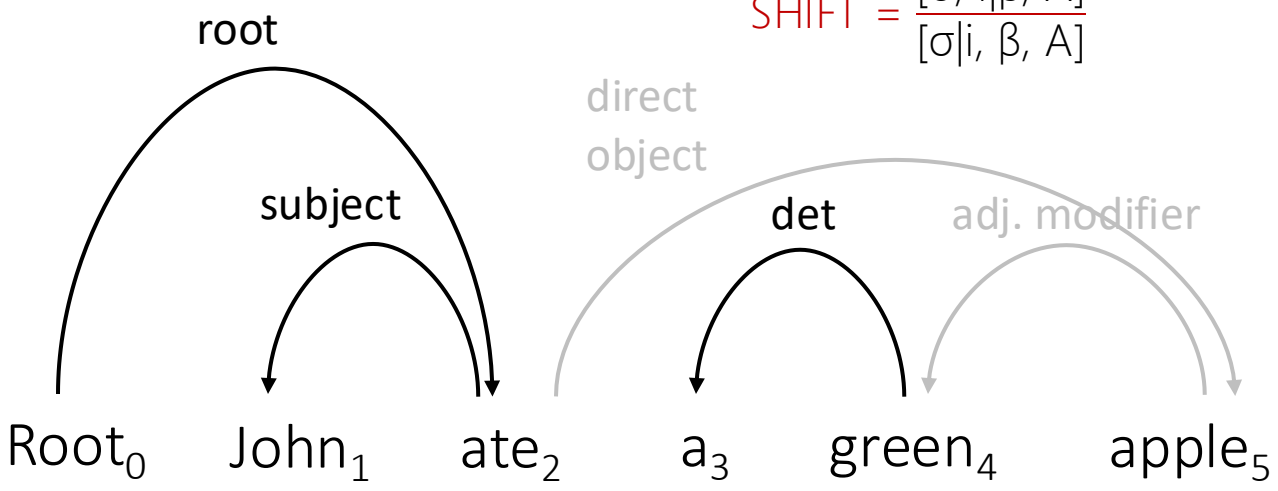
REDUCE Ya se ha creado un arco creado donde la cima del stack (i) es el nodo hijo.

$$\text{LEFT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma, j|\beta, A \cup \{(j, l, i)\}]}$$

$$\text{RIGHT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma|i|j, \beta, A \cup \{(i, l, j)\}]}$$

$$\text{REDUCE} = \frac{[\sigma|i, \beta, A]}{[\sigma, \beta, A]}$$

$$\text{SHIFT} = \frac{[\sigma, i|\beta, A]}{[\sigma|i, \beta, A]}$$



	Stack	Buffer	Action	Arcs
1	[0]	[1,2,3,4,5]	SHIFT	{}
2	[0,1]	[2,3,4,5]	LEFT-ARC	{(2,1)}
3	[0]	[2,3,4,5]	RIGHT-ARC	{(2,1),(0,2)}
4	[0,2]	[3,4,5]	SHIFT	{(2,1),(0,2)}
5	[0,2,3]	[4,5]	LEFT-ARC	{(2,1), (0,2), (4,3)}
6	[0,2]	[4,5]	SHIFT	{(2,1), (0,2), (4,3)}

** Por temas de espacio y claridad, no se indica en la columna Arcs el tipo de dependencia para el arco creado, simplificando como (2,1) el arco (2,subject,1), por ejemplo. En la implementación de la práctica sí es necesario incluir el tipo de dependencia.*

Entrenamiento - Ejemplo de oráculo

Precondiciones

LEFT-ARC La palabra en la cima del stack no es el ROOT(0); y no se ha creado aún un arco donde la cima del stack (i) es el nodo hijo

RIGHT-ARC No se ha creado aún un arco donde el frente del buffer es el nodo hijo

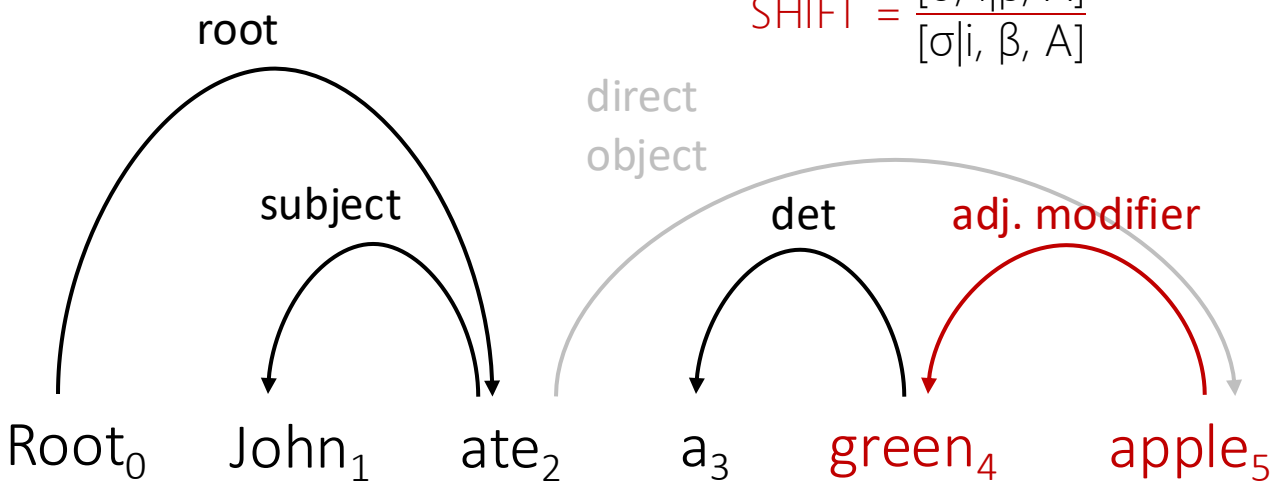
REDUCE Ya se ha creado un arco creado donde la cima del stack (i) es el nodo hijo.

$$\text{LEFT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma, j|\beta, A \cup \{(j, l, i)\}]}$$

$$\text{RIGHT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma|i|j, \beta, A \cup \{(i, l, j)\}]}$$

$$\text{REDUCE} = \frac{[\sigma|i, \beta, A]}{[\sigma, \beta, A]}$$

$$\text{SHIFT} = \frac{[\sigma, i|\beta, A]}{[\sigma|i, \beta, A]}$$



	Stack	Buffer	Action	Arcs
1	[0]	[1,2,3,4,5]	SHIFT	{}
2	[0,1]	[2,3,4,5]	LEFT-ARC	{(2,1)}
3	[0]	[2,3,4,5]	RIGHT-ARC	{(2,1),(0,2)}
4	[0,2]	[3,4,5]	SHIFT	{(2,1),(0,2)}
5	[0,2,3]	[4,5]	LEFT-ARC	{(2,1), (0,2), (4,3)}
6	[0,2]	[4,5]	SHIFT	{(2,1), (0,2), (4,3)}
7	[0,2,4]	[5]	LEFT-ARC	{(2,1), (0,2), (4,3), (5,4)}

** Por temas de espacio y claridad, no se indica en la columna Arcs el tipo de dependencia para el arco creado, simplificando como (2,1) el arco (2,subject,1), por ejemplo. En la implementación de la práctica sí es necesario incluir el tipo de dependencia.*

Entrenamiento - Ejemplo de oráculo

Precondiciones

LEFT-ARC La palabra en la cima del stack no es el ROOT(0); y no se ha creado aún un arco donde la cima del stack (i) es el nodo hijo

RIGHT-ARC No se ha creado aún un arco donde el frente del buffer es el nodo hijo

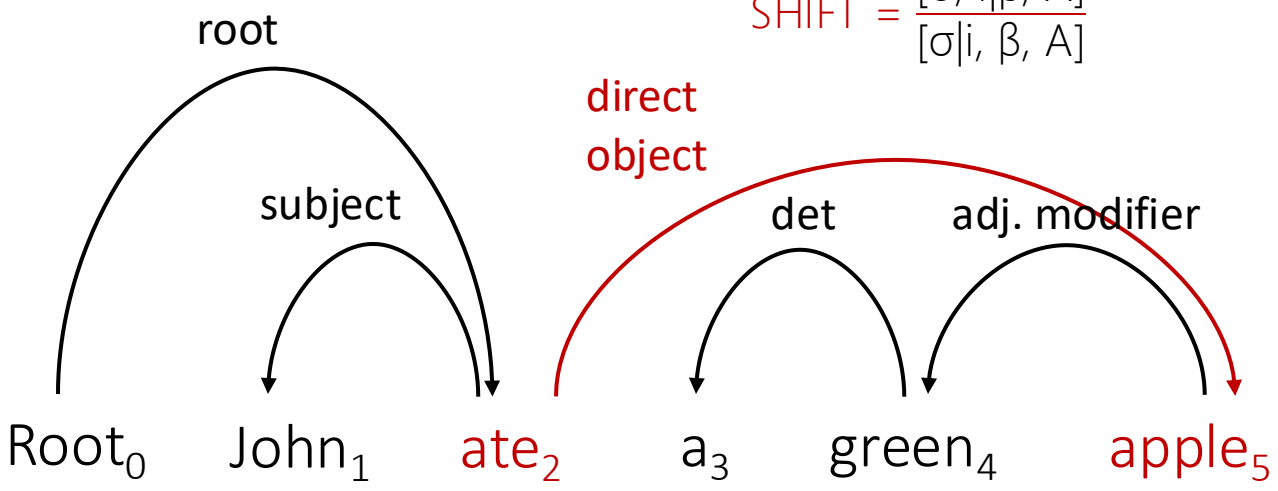
REDUCE Ya se ha creado un arco creado donde la cima del stack (i) es el nodo hijo.

$$\text{LEFT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma, j|\beta, A \cup \{(j, l, i)\}]}$$

$$\text{RIGHT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma|i|j, \beta, A \cup \{(i, l, j)\}]}$$

$$\text{REDUCE} = \frac{[\sigma|i, \beta, A]}{[\sigma, \beta, A]}$$

$$\text{SHIFT} = \frac{[\sigma, i|\beta, A]}{[\sigma|i, \beta, A]}$$



	Stack	Buffer	Action	Arcs
1	[0]	[1,2,3,4,5]	SHIFT	{}
2	[0,1]	[2,3,4,5]	LEFT-ARC	{(2,1)}
3	[0]	[2,3,4,5]	RIGHT-ARC	{(2,1),(0,2)}
4	[0,2]	[3,4,5]	SHIFT	{(2,1),(0,2)}
5	[0,2,3]	[4,5]	LEFT-ARC	{(2,1), (0,2), (4,3)}
6	[0,2]	[4,5]	SHIFT	{(2,1), (0,2), (4,3)}
7	[0,2,4]	[5]	LEFT-ARC	{(2,1), (0,2), (4,3), (5,4)}
8	[0,2]	[5]	RIGHT-ARC	{(2,1), (0,2), (4,3), (5,4), (2,5)}

** Por temas de espacio y claridad, no se indica en la columna Arcs el tipo de dependencia para el arco creado, simplificando como (2,1) el arco (2,subject,1), por ejemplo. En la implementación de la práctica sí es necesario incluir el tipo de dependencia.*

Entrenamiento - Ejemplo de oráculo

Precondiciones

LEFT-ARC La palabra en la cima del stack no es el ROOT(0); y no se ha creado aún un arco donde la cima del stack (i) es el nodo hijo

RIGHT-ARC No se ha creado aún un arco donde el frente del buffer es el nodo hijo

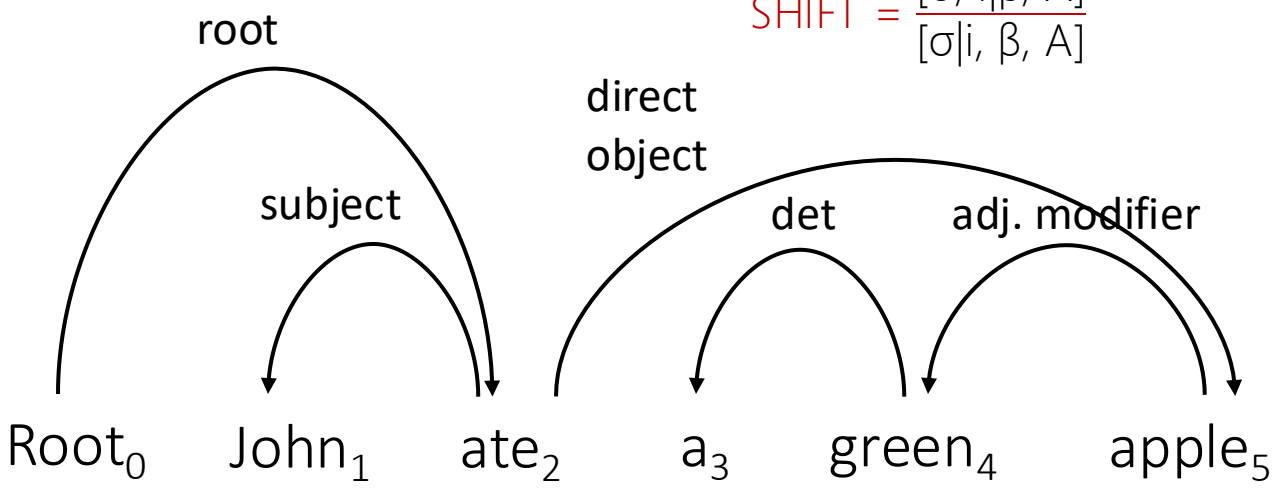
REDUCE Ya se ha creado un arco creado donde la cima del stack (i) es el nodo hijo.

$$\text{LEFT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma, j|\beta, A \cup \{(j, l, i)\}]}$$

$$\text{RIGHT-ARC} = \frac{[\sigma|i, j|\beta, A]}{[\sigma|i|j, \beta, A \cup \{(i, l, j)\}]}$$

$$\text{REDUCE} = \frac{[\sigma|i, \beta, A]}{[\sigma, \beta, A]}$$

$$\text{SHIFT} = \frac{[\sigma, i|\beta, A]}{[\sigma|i, \beta, A]}$$



	Stack	Buffer	Action	Arcs
1	[0]	[1,2,3,4,5]	SHIFT	{}
2	[0,1]	[2,3,4,5]	LEFT-ARC	{(2,1)}
3	[0]	[2,3,4,5]	RIGHT-ARC	{(2,1),(0,2)}
4	[0,2]	[3,4,5]	SHIFT	{(2,1),(0,2)}
5	[0,2,3]	[4,5]	LEFT-ARC	{(2,1), (0,2), (4,3)}
6	[0,2]	[4,5]	SHIFT	{(2,1), (0,2), (4,3)}
7	[0,2,4]	[5]	LEFT-ARC	{(2,1), (0,2), (4,3), (5,4)}
8	[0,2]	[5]	RIGHT-ARC	{(2,1), (0,2), (4,3), (5,4), (2,5)}
9	[0,2,5]	[]	END	{(2,1), (0,2), (4,3), (5,4), (2,5)}

** Por temas de espacio y claridad, no se indica en la columna Arcs el tipo de dependencia para el arco creado, simplificando como (2,1) el arco (2,subject,1), por ejemplo. En la implementación de la práctica sí es necesario incluir el tipo de dependencia.*

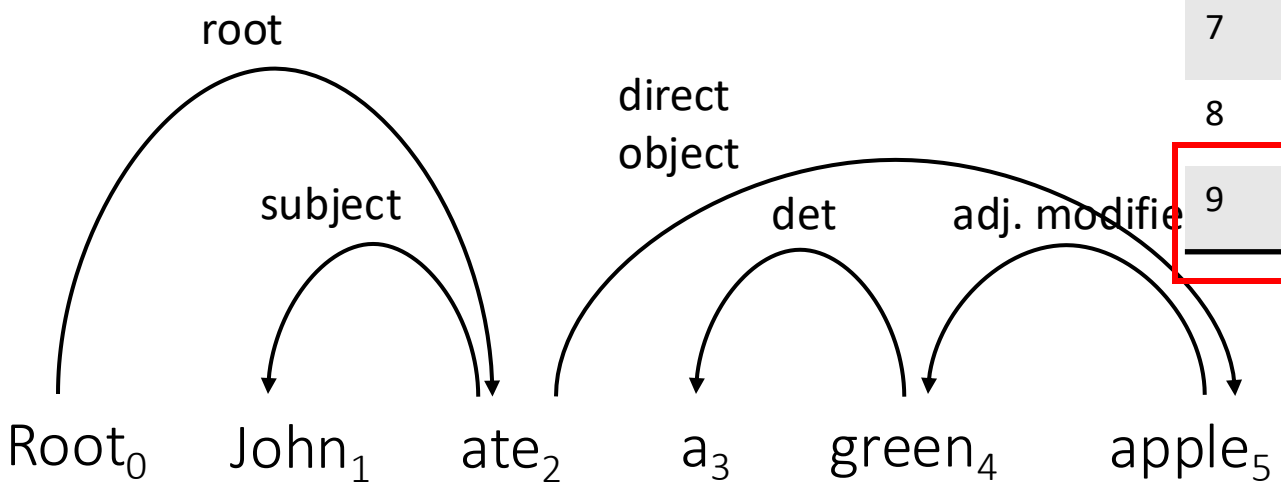
Entrenamiento – Muestras de entrenamiento

Para esta oración el oráculo nos daría 8 muestras de entrenamiento.

Estas muestras son "independientes" entre sí, podemos barajarlas y enviarlas en el orden que queramos al clasificador que vayamos a entrenar.

Lo mismo aplica si las mezclamos con muestra de entrenamiento de otra oraciones

	Stack	Buffer	Salida1 (transición	Salida2 (tipo de dependencia)
1	[0]	[1,2,3,4,5]	SHIFT	None
2	[0,1]	[2,3,4,5]	LEFT-ARC	Subject
3	[0]	[2,3,4,5]	RIGHT-ARC	Root
4	[0,2]	[3,4,5]	SHIFT	None
5	[0,2,3]	[4,5]	LEFT-ARC	Det
6	[0,2]	[4,5]	SHIFT	None
7	[0,2,4]	[5]	LEFT-ARC	Adj. Modifier
8	[0,2]	[5]	RIGHT-ARC	Direct object
9	[0,2,5]	[]		



Estado final, no se considera una muestra de entrenamiento, cuando el buffer queda vacio ya sabemos que hemos acabado.

Entrenamiento – Plantilla de oráculo

Tenéis implementado parte del algoritmo ArcEager en el fichero algorithm.py.

```
class ArcEager():

    """
    Implements the arc-eager transition-based parsing algorithm for dependency parsing.

    This class includes methods for creating initial parsing states, applying transitions to
    these states, and determining the correct sequence of transitions for a given sentence.

    Methods:
        create_initial_state(sent: list[Token]): Creates the initial state for a given sentence.
        final_state(state: State): Checks if the current parsing state is a valid final configuration.
        LA_is_valid(state: State): Determines if a LEFT-ARC transition is valid for the current state.
        LA_is_correct(state: State): Determines if a LEFT-ARC transition is correct for the current state.
        RA_is_correct(state: State): Determines if a RIGHT-ARC transition is correct for the current state.
        RA_is_valid(state: State): Checks if a RIGHT-ARC transition is valid for the current state.
        REDUCE_is_correct(state: State): Determines if a REDUCE transition is correct for the current state.
        REDUCE_is_valid(state: State): Determines if a REDUCE transition is valid for the current state.
        oracle(sent: list[Token]): Computes the gold transitions for a given sentence.
        apply_transition(state: State, transition: Transition): Applies a given transition to the current state.
        gold_arcs(sent: list[Token]): Extracts gold-standard dependency arcs from a sentence.
    """
```

Entrenamiento – Plantilla de oráculo

```
state = self.create_initial_state(sent)

samples = [] #Store here all training samples for sent

#Applies the transition system until a final configuration state is reached
while not self.final_state(state):

    if self.LA_is_valid(state) and self.LA_is_correct(state):
        #Add current state 'state' (the input) and the transition taken (the desired output) to the list of samples
        #Update the state by applying the LA transition using the function apply_transition
        raise NotImplementedError

    elif self.RA_is_valid(state) and self.RA_is_correct(state):
        #Add current state 'state' (the input) and the transition taken (the desired output) to the list of samples
        #Update the state by applying the RA transition using the function apply_transition
        raise NotImplementedError

    elif self.REDUCE_is_valid(state) and self.REDUCE_is_correct(state):
        #Add current state 'state' (the input) and the transition taken (the desired output) to the list of samples
        #Update the state by applying the REDUCE transition using the function apply_transition
        raise NotImplementedError
    else:
        #If no other transiton can be applied, it's a SHIFT transition
        transition = Transition(self.SHIFT)
        #Add current state 'state' (the input) and the transition taken (the desired output) to the list of samples
        samples.append(Sample(state, transition))
        #Update the state by applying the SHIFT transition using the function apply_transition
        self.apply_transition(state,transition)


#When the oracle ends, the generated arcs must
#match exactly the gold arcs, otherwise the obtained sequence of transitions is not correct
assert self.gold_arcs(sent) == state.A, f"Gold arcs {self.gold_arcs(sent)} and generated arcs {state.A} do not match"

return samples
```

Entrenamiento - Plantilla de oráculo

```python
state = self.create_initial_state(sent)

samples = [] #Store here all training samples for sent

#Applies the transition system until a final configuration state is reached
while not self.final_state(state):

    if self.LA_is_valid(state) and self.LA_is_correct(state):
        #Add current state 'state' (the input) and the transition taken (the desired output) to the list of samples
        #Update the state by applying the LA transition using the function apply_transition
        raise NotImplementedError

    elif self.RA_is_valid(state) and self.RA_is_correct(state):
        #Add current state 'state' (the input) and the transition taken (the desired output) to the list of samples
        #Update the state by applying the RA transition using the function apply_transition
        raise NotImplementedError

    elif self.REDUCE_is_valid(state) and self.REDUCE_is_correct(state):
        #Add current state 'state' (the input) and the transition taken (the desired output) to the list of samples
        #Update the state by applying the REDUCE transition using the function apply_transition
        raise NotImplementedError
    else:
        #If no other transiton can be applied, it's a SHIFT transition
        transition = Transition(self.SHIFT)
        #Add current state 'state' (the input) and the transition taken (the desired output) to the list of samples
        samples.append(Sample(state, transition))
        #Update the state by applying the SHIFT transition using the function apply_transition
        self.apply_transition(state,transition)


#When the oracle ends, the generated arcs must
#match exactly the gold arcs, otherwise the obtained sequence of transitions is not correct
assert self.gold_arcs(sent) == state.A, f"Gold arcs {self.gold_arcs(sent)} and generated arcs {state.A} do not match"

return samples
```

Parte del oráculo que debéis implementar

Entrenamiento - Plantilla de oráculo

```python
state = self.create_initial_state(sent)

samples = [] #Store here all training samples for sent

#Applies the transition system until a final configuration state is reached
while not self.final_state(state):

    if self.LA_is_valid(state) and self.LA_is_correct(state):
        #Add current state 'state' (the input) and the transition taken (the desired output) to the list of samples
        #Update the state by applying the LA transition using the function apply_transition
        raise NotImplementedError

    elif self.RA_is_valid(state) and self.RA_is_correct(state):
        #Add current state 'state' (the input) and the transition taken (the desired output) to the list of samples
        #Update the state by applying the RA transition using the function apply_transition
        raise NotImplementedError

    elif self.REDUCE_is_valid(state) and self.REDUCE_is_correct(state):
        #Add current state 'state' (the input) and the transition taken (the desired output) to the list of samples
        #Update the state by applying the REDUCE transition using the function apply_transition
        raise NotImplementedError
    else:
        #If no other transiton can be applied, it's a SHIFT transition
        transition = Transition(self.SHIFT)
        #Add current state 'state' (the input) and the transition taken (the desired output) to the list of samples
        samples.append(Sample(state, transition))
        #Update the state by applying the SHIFT transition using the function apply_transition
        self.apply_transition(state,transition)


#When the oracle ends, the generated arcs must
#match exactly the gold arcs, otherwise the obtained sequence of transitions is not correct
assert self.gold_arcs(sent) == state.A, f"Gold arcs {self.gold_arcs(sent)} and generated arcs {state.A} do not match"

return samples
```

Parte del oráculo implementada: la transición SHIFT

Entrenamiento - Plantilla de oráculo

```python
state = self.create_initial_state(sent)

samples = [] #Store here all training samples for sent

#Applies the transition system until a final configuration state is reached
while not self.final_state(state):

    if self.LA_is_valid(state) and self.LA_is_correct(state):
        #Add current state 'state' (the input) and the transition taken (the desired output) to the list of samples
        #Update the state by applying the LA transition using the function apply_transition
        raise NotImplementedError

    elif self.RA_is_valid(state) and self.RA_is_correct(state):
        #Add current state 'state' (the input) and the transition taken (the desired output) to the list of samples
        #Update the state by applying the RA transition using the function apply_transition
        raise NotImplementedError

    elif self.REDUCE_is_valid(state) and self.REDUCE_is_correct(state):
        #Add current state 'state' (the input) and the transition taken (the desired output) to the list of samples
        #Update the state by applying the REDUCE transition using the function apply_transition
        raise NotImplementedError
    else:
        #If no other transiton can be applied, it's a SHIFT transition
        transition = Transition(self.SHIFT)
        #Add current state 'state' (the input) and the transition taken (the desired output) to the list of samples
        samples.append(Sample(state, transition))
        #Update the state by applying the SHIFT transition using the function apply_transition
        self.apply_transition(state,transition)


#When the oracle ends, the generated arcs must
#match exactly the gold arcs, otherwise the obtained sequence of transitions is not correct
assert self.gold_arcs(sent) == state.A, f"Gold arcs {self.gold_arcs(sent)} and generated arcs {state.A} do not match"

return samples
```

Assert para comprobar que los arcos que generamos con el Oracle son exactamente los que deben ser

Entrenamiento - Extracción de features

Cada par (estado, transición) es una muestra de entrenamiento.

El objetivo es, dada una representación del estado, predecir la acción correcta a tomar.

La etiqueta de salida será la transición predicha (más el tipo de dependencia si es una transición que crea un arco, como por ejemplo el caso del LEFT-ARC o RIGHT-ARC).

¿Cómo representamos el estado para usarlo de entrada a un clasificador?

Entrenamiento - Extracción de features

Cada par (estado, transición) es una muestra de entrenamiento.

Se da implementada la clase Sample dentro de algorithm.py para representar estas muestras de entrenamiento.

```
class Sample(object):
    """
    Represents a training sample for a transition-based dependency parser.

    This class encapsulates a parser state and the corresponding transition action
    to be taken in that state. It is used for training models that predict parser actions
    based on the current state of the parsing process.

    Attributes:
        state (State): An instance of the State class, representing the current parsing
                       state at a given timestep in the parsing process.
        transition (Transition): An instance of the Transition class, representing the
                                 parser action to be taken in the given state.

    Methods:
        state_to_feats(nbuffer_feats: int = 2, nstack_feats: int = 2): Extracts features from the parsing state.
    """
```

Entrenamiento - Extracción de features

Para la representación de la entrada, suelen extraerse un conjunto de features de:

SOLO ES OBLIGATORIO IMPLEMENTAR LAS DE PRIMER ORDEN

- Primer orden: Elementos extraídos directamente del buffer y/o del stack. Por ejemplo, los 2 primeros elementos del buffer y los 2 primeros elementos en la cima del stack.

- Segundo orden: Elementos extraídos a través de relaciones con elementos de primer orden. Por ejemplo, elementos que sean hijos o padre de un determinado elemento del stack. [no se pide en esta práctica]

- Tercer orden: Elementos extraídos en base a dependencias con elementos de segundo orden. Por ejemplo, considerar el hijo izquierdo del hijo más izquierdo de la primera palabra del stack [no se pide en esta práctica]

Entrenamiento - Extracción de features

Para la representación de la entrada, suelen extraerse un conjunto de features de:

- Primer orden: Elementos extraídos directamente del buffer y/o del stack. Por ejemplo, los 2 primeros elementos del buffer y los 2 primeros elementos en la cima del stack.

State: Stack (size=2): (0, ROOT, ROOT_UPOS) | (1, Distribution, NOUN)
Buffer (size=10): (4, license, NOUN) | (5, does, AUX)|...| (13, ., PUNCT)
Arcs (size=2): [(4, 'det', 3), (4, 'case', 2)]

Output: ['ROOT', 'Distribution', 'license', 'does', 'ROOT_UPOS', 'NOUN', 'NOUN', 'AUX']

Entrenamiento – Extracción de features

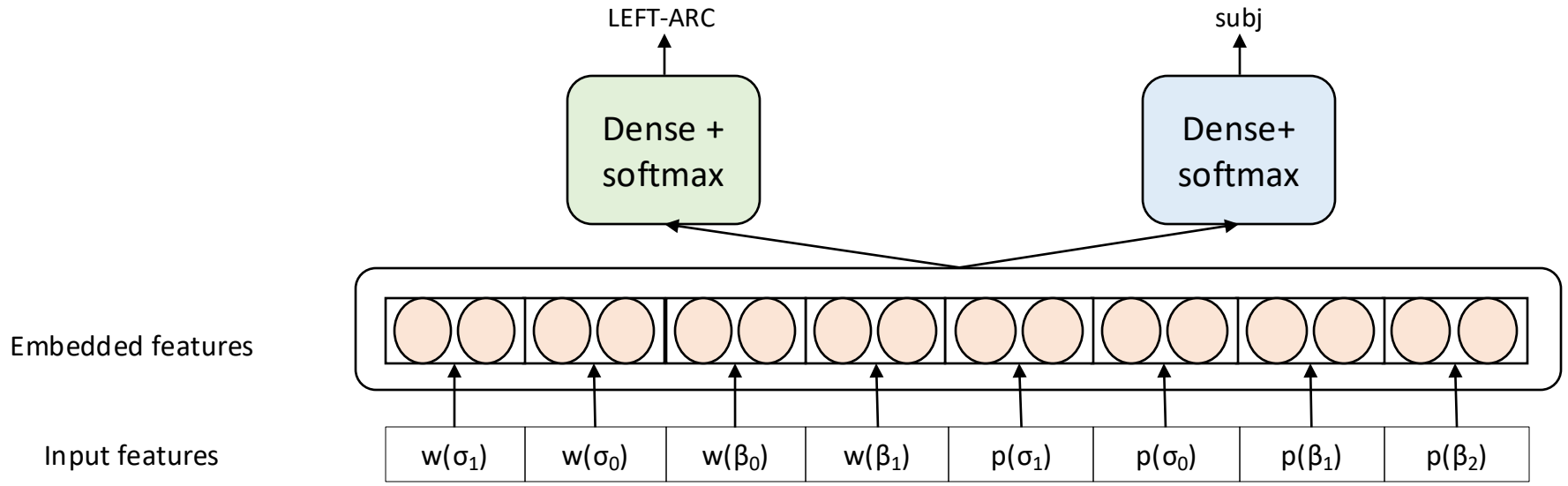
Implementar la función state_to_feats dentro de la clase Sample dentro de algorithm.py, de acuerdo a lo que se indica en los ejemplos

```python
def state_to_feats(self, nbuffer_feats: int = 2, nstack_feats: int = 2):
    """
    Extracts features from a given parsing state for use in a transition-based dependency parser.

    This function generates a feature representation from the current state of the parser,
    which includes features from both the stack and the buffer. The number of features from
    the stack and the buffer can be specified.

    Parameters:
        nbuffer_feats (int): The number of features to extract from the buffer.
        nstack_feats (int): The number of features to extract from the stack.

    Returns:
        list[str]: A list of extracted features. The features include the words and their
                corresponding UPOS (Universal Part-of-Speech) tags from the specified number
                of elements in the stack and buffer. The format of the feature list is as follows:
                [Word_stack_n,...,Word_stack_0, Word_buffer_0,...,Word_buffer_m,
                    UPOS_stack_n,...,UPOS_stack_0, UPOS_buffer_0,...,UPOS_buffer_m]
                where 'n' is nstack_feats and 'm' is nbuffer_feats.

    Examples:
        Example 1:
            State: Stack (size=1): (0, ROOT, ROOT_UPOS)
                Buffer (size=13): (1, Distribution, NOUN) | ... | (13, ., PUNCT)
                Arcs (size=0): []

            Output: ['<PAD>', 'ROOT', 'Distribution', 'of', '<PAD>', 'ROOT_UPOS', 'NOUN', 'ADP']

        Example 2:
            State: Stack (size=2): (0, ROOT, ROOT_UPOS) | (1, Distribution, NOUN)
                Buffer (size=10): (4, license, NOUN) | ... | (13, ., PUNCT)
                Arcs (size=2): [(4, 'det', 3), (4, 'case', 2)]

            Output: ['ROOT', 'Distribution', 'license', 'does', 'ROOT_UPOS', 'NOUN', 'NOUN', 'AUX']
    """
    raise NotImplementedError
```

Entrenamiento – Modelo

Concatenamos las representaciones de entrada y predecimos la siguiente transición.

En esta práctica simplemente extraeremos las features requeridas, las representaremos como vectores y las concatenaremos como entrada a una capa de clasificación con feed-forward networks y multi-task learning.



En el ejemplo, donde solo estamos usando features de primer orden, *w* es una función que extrae la forma de la palabra de un elemento de buffer o del stack, y *p* es una función que extrae la correspondiente etiqueta morfológica.

Predicción – “en vertical” [eficiente]

1. Coger un batch de oraciones de entrada.
2. Obtener el correspondiente batch de estados iniciales.
3. Extraer el batch de features para el conjunto de estados actuales.
4. Realizar la predicción con el modelo entrenado a nivel de batch.
5. Verificar que las transiciones predichas cumplen las precondiciones.
6. Actualizar los estados.
7. Eliminar aquellas instancias del batch que hayan alcanzado un estado final.
8. Repetir desde el punto 3 hasta que el batch se quede vacío.

Ejemplo Predicción – "en vertical" [eficiente] con un conjunto de 2 oraciones (en la práctica será el total del conjunto de test)

1. Coger el batch the oraciones:

batch_sentences = ["John ate a green apple", "Mary is sick"]

2. Obtener sus estados iniciales:

batch_states= [(Stack=ROOT, Buffer = John|ate|a|green|apple, Arcs={}) , (Stack=ROOT, Buffer=Mary|is|sick), Arcs={}]

3. Transformar el estado en el conjunto de features que lo representan:

batch_state_feats= [[<PAD>, ROOT, John, ate, <PAD>, ROOT_UPOS, NOUN, VERB],
[<PAD>, ROOT, Mary, is, <PAD>, ROOT_UPOS, NOUN, VERB]]

4. Llamar al modelo para que obtenga la distribución de probabilidad para cada posible transición y los tipos de dependencias:

batch_actions, batch_deprels = model.predict(batch_state_feats)

Ejemplo Predicción – "en vertical" [eficiente] con un conjunto de 2 oraciones (en la práctica será el total del conjunto de test)

5. Seleccionar el tipo de dependencia más probable, y la transición válida más probable (es decir que respeta las pre-condiciones impuestas por el Arc-eager). Por ejemplo, suponiendo que ya tenemos las transiciones ordenadas por probabilidad, podríamos tener algo cómo:

probs_transitions= [[SHIFT (0.4), RA (0.3), LA (0.2), REDUCE (0.1)], [~~LA (0.3),~~ SHIFT(0.25), REDUCE (0.25), RA (0.2)]]

6. Aplicar la transición seleccionada a nuestros estados:

batch_selected_transitions (SHIFT, SHIFT)

batch_states = [(Stack=ROOT, Buffer = John|ate|a|green|apple, Arcs={}) , (Stack=ROOT, Buffer=Mary|is|sick), Arcs={}]

new_states = [(Stack=ROOT|John, Buffer = ate|a||green|apple, Arcs={}) , (Stack=ROOT|Mary Buffer=is|sick), Arcs={}]

batch_states = new_states

7. Se eliminan del batch las oraciones con el Buffer vacío (en este caso no hay ningún elemento del batch que cumpla esta condición) y volvemos al paso 3.

Ejemplo Predicción - Salida

Para poder evaluar la precisión de nuestro modelo sobre el conjunto de test, debemos primero escribir la salida en un fichero en formato CoNLL-U.

Para ello debemos implementar una función que transforme la secuencia de transiciones a un árbol de dependencias en formato CoNLLU. La función es similar a la función del oráculo.

Al hacer las predicciones, el estado final almacena todos los arcos creados.

Con esos arcos y las palabras de entrada, podemos generar el árbol.

Postprocesado (ya implementado en postprocessor.py)

Es posible que al hacer predicciones obtengamos un árbol corrupto. Deberán corregirse con heurísticas para obtener árboles que cumplan los requisitos.

Algunos ejemplos de situaciones que causan árboles malformados:

- Nodos sin un padre asignado (es fácil pensar en un caso extremo, una secuencia de transiciones en la que solo hacemos SHIFT)

- Oraciones donde a ningún término se le asigna como padre el ROOT.

- Oraciones donde a varios términos se les asigna como padre el ROOT. Esto no es estrictamente incorrecto, pero en UD se considera que los árboles deben ser *single-rooted*. Por tanto debéis controlarlo, o habrá problemas en la evaluación.

- En el arc-eager no hay posibilidad de que se formen ciclos, pero en otros algoritmos de transiciones si es posible.

Postprocesado (ya implementado en postprocessor.py)

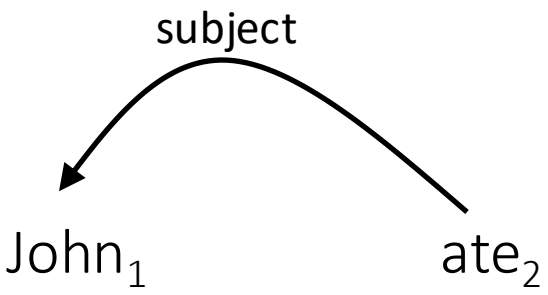
El módulo de postprocesado también se da implementado dentro de la clase Postprocessor en postprocessor.py. Se incluye un ejemplo de uso al final del fichero.

```
class PostProcessor():
    """
    A class for post-processing syntactic trees parsed from CoNLL-U formatted files.

    This class provides functionality to correct trees that have issues such as
    multiple roots or tokens without an assigned head. It ensures that each tree
    conforms to a standard structure with a single root and all tokens having
    an assigned head.

    Methods:
        postprocess: Corrects the trees in a given CoNLL-U file and returns the corrected trees.
    """
```

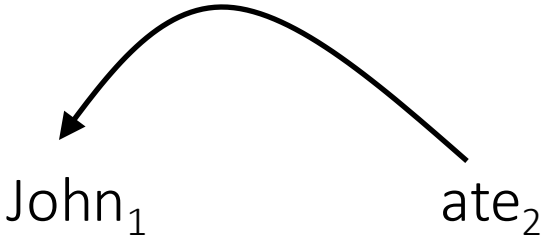
Evaluación

SEGÚN EL PROFESOR, UN LAS DE 60% ES CORRECTO

LAS (Labeled Attachment Score): El porcentaje de palabras para los que se acierta el HEAD y el DEPREL.



UAS (Unlabeled Attachment Score): El porcentaje de palabras para los que se acierta el HEAD, pero no el DEPREL.



Para la evaluación, deberéis usar el script oficial conll18_ud_eval.py, disponible en: https://universaldependencies.org/conll18/evaluation.html